Project 12.1 — ComplyDesk-Policy-Checked-Support-Replies-As-One-MCP-Tool

A practice twin of Project 12 — MCPResearch. Same skills — a generate→verify→revise pipeline (two agents, capped loop) wrapped behind **one** MCP tool (the "agent-as-a-service" pattern), with a progress trace and a timeout/partial fallback. A **new, real-world job**. **There are no steps here on purpose.** Read the story, build it yourself, and only open the hints or the solution if you get stuck.

Type: MCP Server (1 Tool, wrapping a 2-agent pipeline) · **Stack:** Python, `mcp` SDK (MCPServer), `langchain-openai`, asyncio, Pydantic · **Level:** Advanced

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A support team wants their chatbots and tools to draft replies to customer tickets — but a reply that promises a refund the agent isn't authorized to give, or "guarantees" something the company can't, is worse than no reply. They don't want every tool that drafts a reply to re-implement the compliance rules. They want **one** function they can call: give it a ticket, get back a reply that's already been checked against policy.

You are going to build **ComplyDesk**: that capability as a single MCP tool. Underneath is a two-agent loop — but the caller never sees it:

- A **Drafter** writes a friendly, helpful reply to the customer's ticket.
- A **Compliance-Checker** rejects it with specific issues if it breaks policy: promising an unauthorized refund (agents may offer a 10% credit; full refunds need manager approval), using banned phrases ("I guarantee", "I promise"), or leaking internal information.
- The Drafter rewrites, fixing exactly those issues — bounded by a hard round cap so a never-satisfied checker can't loop forever.

All of that is hidden behind one `draft_safe_reply(ticket) -> str` MCP tool. Any client calls it and gets back a policy-safe reply plus a short trace of what was fixed. And because a multi-round call can be slow or break, ComplyDesk has a **timeout + partial fallback**: if it runs too long, it returns the best draft so far, clearly marked — never a hang.

The one lesson of this project: you can offer a whole generate-then-verify workflow as one callable capability over a standard protocol, so other teams' tools use it without re-implementing (or

depending on) its internal steps — and *you* decide what that slow, multi-step process looks like from the outside: a trace, a timeout, a partial.

How it works, from a ticket to a safe reply (picture the whole machine)

Read this once and the whole project should form in your head.

A client calls the `draft_safe_reply` MCP tool with a ticket. Inside, a synchronous pipeline runs a loop: the **Drafter** (a prompt → model call) writes a reply; the **Compliance-Checker** (a prompt → `model.with_structured_output`) returns a checked verdict — `approved` plus a list of specific `issues`. If approved, the loop stops. If not, the issues are fed back to the Drafter as feedback (fix *only* these), and it drafts again — up to a hard `MAX_REVISION_ROUNDS`, then it stops regardless. A `Progress` holder records the latest draft and each round as it goes.

The MCP tool is `async`, but the pipeline is synchronous, so the tool runs it with `asyncio.to_thread(run_pipeline, ...)` wrapped in `asyncio.wait_for(..., timeout)` — running it on a worker thread keeps the event loop free so the timeout can actually fire (wrapping a blocking call directly in `wait_for` would just block). On success, the tool returns a formatted block: the final reply, a status, and a **revision trace** showing what the compliance checker caught and the drafter fixed. On timeout, it returns the best draft so far from the `Progress` holder, clearly marked PARTIAL.

That's the whole machine: **one MCP tool → a Drafter/Compliance-Checker loop (capped, with feedback) on a worker thread → a policy-safe reply with a trace → a marked partial on timeout, never a hang.**

What you are building (the pieces)

- A **Drafter** (`run_drafter(model, ticket, feedback=None)`) and a **Compliance-Checker** (`run_compliance(model, draft) -> ComplianceVerdict`), with real policy criteria in its prompt.
- A **pipeline** (`run_pipeline`) that loops them, capped, recording progress.
- A **formatter** (`format_result`, `format_partial`) for the trace and the partial.
- One **MCP tool** (`draft_safe_reply`) that runs the pipeline with a timeout and falls back to a partial.

Inputs: a customer ticket. **Outputs:** a policy-safe reply with a revision trace; or a marked partial on timeout. **Rules:** the layers stay separate (drafter / checker / pipeline / MCP); the loop is capped; the tool always returns something.

A real example to build and test against

- Ticket: *"My order #1001 arrived late — I want a full refund."* → round 1 the Drafter promises a full refund; the Compliance-Checker rejects it ("agents may offer a 10% credit; full refunds need manager approval"); round 2 offers the credit and escalation path, and is approved.
- A ticket with nothing to flag → approved on round 1.
- A ticket you deliberately make slow on → the timeout returns a marked PARTIAL, not a hang.

How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 7's MCP server and Project 4's Writer/Reviewer loop if hazy.)
2. Plan it on paper: which layer owns which job — the Drafter, the Compliance-Checker, the pipeline (loop + cap), the MCP tool (format, timeout, fallback). Don't put the timeout inside the Drafter.
3. Build it yourself. A good order: the two-agent pipeline standalone → wrap it as one MCP tool → add the trace → add the timeout + partial. Your call.
4. Test it on the real example: a reply that gets fixed, a clean ticket, and a forced-slow timeout.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

Offering a whole workflow as one protocol-callable capability is a real, increasingly common production pattern — a policy-checked reply here, but the same shape fits a "generate SQL and validate it" tool, a "draft and brand-check copy" tool, a "summarize and fact-check" tool. The skill isn't the agents; it's deciding what a slow, multi-step process should look like from the outside, so other teams can depend on the *capability* without depending on its internals. The trace keeps it from being a black box; the timeout + partial keeps it from ever hanging. Build ComplyDesk and you can wrap any generate-then-verify workflow as a clean, dependable service.

Done when

- The pipeline was proven standalone before any MCP wrapped it.
- `draft_safe_reply` is one tool whose body runs the pipeline; the result includes a revision trace.
- The loop stops at `MAX_REVISION_ROUNDS` on a ticket the checker never approves.
- A slow run returns a marked PARTIAL, not a hang (`to_thread` + `wait_for`).
- Your tests pass: settings, the formatter, the loop + cap (fake model), and a live `draft_safe_reply` call.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.